

The Abstraction Gap: Human Language, Machine Language, and the Future of Programming Harnesses

A Deep Research Brief on the Spectrum Between Natural Language and Code, From Dijkstra to AI Coding Agents

Compiled: April 2, 2026 Lead Researcher: Claude Opus 4.6

Executive Summary

In March 2026, Gabriel Gonzalez published "A sufficiently detailed spec is code," invoking Dijkstra's 1978 essay EWD 667 to argue that the dream of spec-driven AI coding inevitably collapses: as specifications become precise enough to be unambiguous, they converge on code itself. This research brief investigates the deeper question: **how much abstraction should exist between human language and machine language, and what do the foundational thinkers of computer science, human-computer interaction, and cognitive science tell us about how future harnesses and systems should mediate this boundary?**

We surveyed viewpoints from 40+ major thinkers across three intellectual traditions, analyzed 6 current AI coding tools, reviewed empirical benchmarks and practitioner reports, and examined intermediate representations from formal methods to dependent types. Our findings:

1. **The formalist tradition (Dijkstra, Hoare, Church, Martin-Lof)** proves mathematically that sufficiently precise specifications converge on code. The Curry-Howard correspondence demonstrates this is not a practical observation but a theorem: proofs ARE programs, propositions ARE types.
2. **The pragmatist middle ground (Brooks, Parnas, Knuth, Lamport)** argues that the spec-code gap serves a purpose -- abstraction, information hiding, separation of concerns -- and collapsing it destroys useful structure. A spec that is detailed enough to be code has *failed as a spec*.

3. **The contrarian position (Karpathy, Suchman, Clark, Wittgenstein)** makes a surprisingly strong case: LLMs solved the parsing problem Dijkstra identified; cognition is distributed across human+tool systems; meaning is constituted by use, not formal structure; and the abstraction stack has always moved upward.
4. **The empirical evidence** is sobering: SWE-bench scores reach 77%, but ~50% of passing PRs would not be merged by maintainers (METR, 2026). HumanEval is saturated at 90%+, but LiveCodeBench shows only 38.9% on real-world tasks. Code quality metrics show 17% maintainability drops and rising bug rates over 6 months.
5. **The most promising path forward** is a "third way" -- intermediate representations (TLA+, dependent types, contracts, property-based specifications) where humans author specifications at an intermediate formalism level, AI generates implementations and proofs, and formal verifiers provide deterministic correctness guarantees. The "vericoding" pattern already achieves 82% success on Dafny specifications.

For harness design, the implications are concrete: harnesses should operate at the specification level, support multiple levels of formalism, use AI as a "formalism translator," employ automatic verification, and preserve the theory (reasoning, rationale, context) behind decisions.

Part I: The Formalist Tradition -- Why Precision Demands Formalism

1.1 Dijkstra: "The Foolishness of Natural Language Programming"

Edsger Dijkstra's EWD 667 (1978) is the intellectual anchor for the entire debate. His argument comprises five interlocking claims:

The Interface Fallacy. Changing the interface between human and machine does not reduce total work; it redistributes it. Making the interface "natural" for humans makes it harder for the machine, and total complexity is conserved or increased.

The Historical Precedent. "Greek mathematics got stuck because it remained a verbal, pictorial activity." Progress came only with formal symbolic notation (Descartes, Leibniz). Formal notation is not an obstacle to thought but a *precondition* for it.

The Virtue of Formalism. "The virtue of formal texts is that their manipulations, in order to be legitimate, need to satisfy only a few simple rules." Natural language lacks this mechanical checkability.

The Insidiousness of Naturalness. The "naturalness" of natural language "boils down to the ease with which we can use them for making statements the nonsense of which is not obvious." Natural language makes it *easy* to say things that sound meaningful but are vacuous.

Formalism as Democratization. Formal notation enables "ordinary people to accomplish what once required genius." The notation does the heavy lifting that unaided human cognition cannot.

A decade later, in EWD 1036 ("On the cruelty of really teaching computing science"), Dijkstra deepened his position: computing represents a "radical novelty" where "the smallest possible perturbations can have the most drastic consequences." Programs are "abstract formulae" requiring formal proof, and he proposed teaching programming using languages *deliberately unimplemented* to force mathematical rigor over experimentation.

Dijkstra's verdict on the spec-vs-code question: A sufficiently precise specification IS code -- not because specs "converge" on code, but because precision *requires* formalism, and formalism *is* programming. The desire for natural-language specification is the desire to be imprecise, which is the desire to not actually specify [1, 2].

1.2 Hoare: The Axiomatic Bridge

Tony Hoare's axiomatic semantics (1969) established the formal bridge between specification and code. The Hoare triple $\{P\} C \{Q\}$ -- if precondition P holds before executing command C , then postcondition Q holds afterward -- places specification and code in the *same formal universe*.

"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies." -- "The Emperor's Old Clothes," Turing Award Lecture, 1980

Hoare's position: specifications and programs are both formal objects. A specification that is precise enough to verify is precise enough to be (or generate) code. The gap between spec and code is a gap of *detail*, not of *kind* [5, 6].

1.3 The Curry-Howard-Martin-Lof Correspondence: Spec IS Code, by Theorem

Per Martin-Lof's intuitionistic type theory (1972-1984) provides the mathematical culmination of the formalist position. In this system:

- **Types are propositions** (specifications)
- **Terms (programs) are proofs** (implementations)
- **Type checking is proof verification** (mechanical and decidable)

Martin-Lof stated: "It is the contention of the intuitionists that the basic mathematical notions ought to be interpreted in such a way that the cleavage between mathematics and programming disappears."

Philip Wadler's "Propositions as Types" (2015) traces this through its full history and concludes it is not an analogy but a mathematical *identity*: proofs literally ARE programs, and propositions literally ARE types [20, 39].

Logic	Programming
Propositions	Types
Proofs	Programs
Implication ($A \rightarrow B$)	Function type ($A \rightarrow B$)
Conjunction ($A \wedge B$)	Product type ($A \times B$)
Proof simplification	Program evaluation

This is the deepest formal result in the debate: in sufficiently expressive type systems, the distinction between specification and implementation is a matter of *perspective*, not of *kind*.

1.4 Information-Theoretic Limits

Kolmogorov complexity and algorithmic information theory place hard mathematical constraints on the spec-code relationship:

- **For incompressible programs** (most programs, by counting), the shortest specification is approximately equal to the program itself. Gonzalez's thesis is *mathematically necessary* for complex software.
- **Chaitin's incompleteness theorem** proves that an N-bit formal system can only specify programs of complexity at most N. To specify more complex programs, the specification framework must grow -- and in the limit, *is* the program.
- **Rice's theorem** establishes that verifying whether code satisfies a spec is undecidable in the general case, meaning that even when spec and code converge in length, they remain semantically distinct.

These results prove that spec-code convergence is real but undetectable in the general case (Kolmogorov uncomputability), and that verification of the convergence is undecidable (Rice) [35, 36, 37].

Part II: The Pragmatist Middle Ground -- Why the Gap Serves a Purpose

2.1 Brooks: The Specification IS the Hard Part

Fred Brooks's "No Silver Bullet" (1986) distinguishes essential complexity (inherent in the problem) from accidental complexity (artifacts of tools and processes):

"I believe the hard part of building software to be the specification, design, and testing of this conceptual construct, not the labour of representing it and testing the fidelity of the representation."

If the specification IS the hard part, and a sufficiently detailed specification IS code, then specification IS programming. AI tools that promise to eliminate "coding" are actually promising to eliminate the easy part while leaving the hard part untouched [8].

2.2 Parnas: Information Hiding Requires the Gap

David Parnas's information hiding principle (1972) argues that a good specification describes *what* without *how*. Module boundaries should be drawn around design decisions likely to change. A spec that is detailed enough to be code has *failed* as a specification because it has revealed implementation decisions that should have been hidden [9].

2.3 Knuth: The Hybrid Document

Donald Knuth occupies a unique position: a deep formalist who argued that programs should be written primarily for humans to read. His literate programming paradigm (1984) interleaved natural language explanation with formal code:

"Let us change our traditional attitude to the construction of programs: Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer to do."

Knuth's resolution: both natural language and formal notation are necessary. Natural language provides context, motivation, and narrative; formal notation provides precision and executability. Neither alone is sufficient [7].

2.4 Lamport: Thinking Above the Code

Leslie Lamport's career-spanning argument is that *thinking above the code* is the essential discipline. In "Thinking Above the Code," he distinguishes: (1) deciding *what* the program should do, (2) deciding *how* (the algorithm), and (3) encoding in a programming language. Most bugs originate in (1) and (2), yet the profession obsesses over (3).

Lamport explicitly resists making TLA+ more like a programming language. In "The Future of TLA+" (2024), he argues that requests to make specs "more like code" are a *hindrance*: "the problem is thinking programmatically instead of thinking outside of that box. And the way to think outside of that box is to think mathematically" [Lamport, Quanta, 2022].

2.5 Naur: Programming as Theory Building

Peter Naur's 1985 paper presents the most radical challenge to the entire specification enterprise. His claim: a program embodies a *theory* -- tacit knowledge about how the world maps to the program, which parts matter, and why decisions were made. This theory *cannot be fully externalized* in specs, docs, or code.

If Naur is right, no intermediate representation -- however formal -- can fully bridge the abstraction gap. The theory lives in developers' minds, and a program's health is "entangled with the continuity of the people who hold its theory." This challenges both the formalist vision (just write a perfect spec) and the NL vision (just tell the AI what you want) [Naur, 1985].

Part III: The Cognitive Science Constraint -- How Human Minds Process Specifications

3.1 Kahneman: System 1 vs System 2 and the Danger of "Naturalness"

Daniel Kahneman's dual-process theory maps directly onto the spec debate. Programming demands System 2 thinking: careful, deliberate, logical reasoning. System 1 -- fast, intuitive, pattern-matching -- is the source of bugs.

Natural language specification *invites* System 1 processing: it reads smoothly, "makes sense," feels complete. This is exactly Dijkstra's concern about making "statements the nonsense of which is not obvious." Formal specification *forces* System 2 engagement: each symbol must be parsed, each relationship verified. The difficulty of reading formal notation is a *feature*, not a bug -- it prevents System 1 from generating the illusion of understanding [33].

3.2 Pinker: Mentalese and the Lossy Encoding of Thought

Steven Pinker argues that human thought is conducted not in natural language but in "mentalese" -- a deeper representational system. Natural language is a *lossy encoding* of thought: when we translate thought into words, we lose precision. When we translate words back, we must reconstruct what was lost.

The implication: natural language specifications are *doubly lossy* -- thought -> natural language (loss #1) -> computer interpretation (loss #2). Formal programming languages may be *closer to mentalese* than natural language is, capturing structural and logical aspects more faithfully [31].

3.3 Chomsky: The Competence/Performance Distinction

Chomsky's distinction between competence (the ideal speaker's knowledge of the language -- a formal system) and performance (actual language use -- noisy, context-dependent) maps onto specs vs. code. Linguistics should study competence, not performance; analogously, Dijkstra insisted computer science should study formal specifications, not heuristic implementations.

Chomsky's 2011 dismissal of statistical NLP -- "it interprets success as approximating unanalyzed data" -- presages the formalist critique of LLM-based coding: producing code that works is not the same as understanding why it works [29, 30].

3.4 Miller: The Magical Number Seven and Chunking

George Miller's discovery that working memory holds approximately 7 ± 2 chunks has direct implications: any specification must be decomposable into manageable chunks. Natural language specifications are poorly chunked (sentences don't correspond to logical units); formal specifications enforce explicit chunking through type signatures, function signatures, and module interfaces [32].

3.5 Sapir-Whorf Applied to Programming: Notation as a Tool of Thought

The weak Sapir-Whorf hypothesis -- that language structure influences perception -- has direct programming applications:

- **Paul Graham's "Blub Paradox"**: A programmer in a less powerful language cannot perceive features they lack, because they have no concepts for them.
- **Kenneth Iverson's Turing Award lecture "Notation as a Tool of Thought"** (1979): APL's notation enabled mathematical thoughts impossible in conventional notation.
- **Dijkstra's observation** that different notations make different thoughts possible is itself a Whorfian claim.

If the language shapes thought, then the *specification language* shapes what can be specified. Natural language inherits all the conceptual limitations (and affordances) of natural language. Formal notation may enable thoughts about invariants, concurrency, types, and proofs that natural language makes difficult [34, 42].

Part IV: The Contrarian Case -- Why Natural Language Might Work After All

4.1 Karpathy: English as the New Programming Language

Andrej Karpathy's argument evolved through three phases:

- **Software 2.0 (2017):** Neural networks are a new kind of software where "code" is weights, not instructions.
- **"English is the hottest new programming language" (2023):** If LLMs translate NL to code, English IS a programming language.
- **Vibe coding to agentic engineering (2025-2026):** From "forget the code exists" to disciplined orchestration of AI agents.

Karpathy's steel-manned response to Dijkstra: the critique targeted a world where machines could not parse natural language. That constraint no longer holds. The question is not whether NL is *formally precise* but whether it is *functionally adequate* as an interface to generation systems that handle the translation [Karpathy, 2017-2025].

4.2 Suchman: Situated Actions Destroy the Planning Model

Lucy Suchman's "Plans and Situated Actions" (1987) argued that the dominant AI/cognitive science view treats plans as "prerequisite to and prescribing action." She showed this model fails: human action is "necessarily ad hoc responses to the actions of others and to the contingencies of particular situations."

Applied to programming: the waterfall model (write spec -> implement -> test) IS the planning model. It fails because specifications cannot anticipate situated contingencies. NL-mediated programming with LLMs is *radically situated*: the developer converses, evaluates, adjusts, and refines. This is closer to Suchman's situated actions than to formal planning. The formalist critique of NL programming (too imprecise for plans) misidentifies the problem: programming was never well-served by the planning model [Suchman, 1987].

4.3 Clark and Hutchins: The Extended and Distributed Mind

Andy Clark's extended mind thesis (1998) argues that cognition extends into tools. A developer working with an LLM is not outsourcing cognition but *extending* their cognitive system. The human-LLM system is a cognitive unit with capabilities neither has alone.

Edwin Hutchins' distributed cognition framework takes this further: navigation on a ship is an emergent property of crew + instruments + charts + procedures, not of any individual. Similarly, human + LLM + IDE + tests + version control constitutes a distributed cognitive system. The system achieves precision even if no component (including the human's NL specification) is precise on its own [Clark, 1998; Hutchins, 1995].

4.4 Wittgenstein: Meaning Is Use, Not Formal Structure

Wittgenstein's later philosophy (Philosophical Investigations, 1953) argues that meaning is not determined by formal structure but by *use in context* -- "language games." A prompt is a move in a language game. Its meaning is determined not by formal semantics but by the *practice* of prompting -- conventions, expectations, and iterative refinements.

The Dijkstra position assumes meaning must be formally specified to be precise. Wittgenstein showed that meaning can be practically determinate *without* formal specification -- through shared forms of life. NL programming creates new language games that can be effective without formal specification [Wittgenstein, 1953].

4.5 The Seven Specific Rebuttals

Rebuttal	Strongest Source	Core Argument
LLMs solve the parsing problem	Brooker (Amazon VP)	"Almost all programs are already specified in NL. LLMs automate the translation."
Abstraction always moves up	Historical pattern	Assembly -> C -> Python -> English follows the same trajectory every previous generation resisted.
Good enough beats perfect	Narayanan (Princeton)	Formal correctness matters for ~1% of software; accessibility matters for 100%.
The spec IS the program	Syme (F# creator)	"A program is instructions given to a machine to achieve results." A prompt that produces working code IS a program.
Conversational refinement	Interactive agents research	Each turn transfers information, reducing entropy. Total information can equal a formal spec.
Programming is communication	Knuth, Carmack	"'Coding' was never the source of value. Problem solving is the core skill."
Democratization	185:1 ratio	5B NL speakers vs 27M programmers. NL programming is a 100x expansion of global capability.

4.6 Sutton's Bitter Lesson

Rich Sutton's "The Bitter Lesson" (2019) predicts that general methods scaling with computation outperform domain-specific engineering. Applied to programming: attempts to engineer precise specification languages may be outcompeted by training massive language models on all available code. Dijkstra's approach -- designing formal languages for formal thought -- is the *epitome* of the approach the Bitter Lesson predicts will lose. Not because it is wrong, but because it does not scale [Sutton, 2019].

Part V: The Empirical Reality -- What the Data Actually Shows

5.1 Benchmarks Tell a Split Story

Benchmark	Top Score	What It Measures	Gap Indicator
HumanEval	90%+	Isolated function generation from docstrings	Saturated -- too easy
MBPP	90%+	Similar isolated tasks	Saturated
SWE-bench Verified	77.2%	Real GitHub issues	METR: ~50% of passes wouldn't be merged
LiveCodeBench	38.9%	Dynamic real-world tasks	Still very challenging
ARC-AGI-2	~30%	Novel reasoning tasks	Genuinely hard

The gap between HumanEval (90%+) and LiveCodeBench (38.9%) *quantifies the abstraction gap*. Isolated function generation from clear NL descriptions is nearly solved. Real-world software engineering is far from it [SWE-bench, METR 2026, LiveCodeBench].

5.2 The METR Reality Check

METR's March 2026 study is the most important empirical result: maintainers of scikit-learn, Sphinx, and pytest reviewed 296 AI-generated PRs that passed all tests. Key findings:

- **~50% would not be merged** despite passing automated tests
- Quality problems: code quality issues, undocumented failures, broken code, core functionality failure
- **Human-written patches had 68% merge rate** (the baseline)
- Agents were not given a chance to iterate -- the gap might narrow with feedback

This confirms Brooks's insight: passing tests is not the same as satisfying the specification. The specification includes tacit quality standards, architectural conventions, and design intent that no test suite captures [METR, 2026].

5.3 Code Quality Degradation

GitClear's 2025 study (211M changed lines, 2020-2024) found: - Refactoring dropped from 25% to <10% of changed lines - Copy-pasted code rose from 8.3% to 12.3% - Maintainability index dropped 17% - Bugs: 19% lower short-term, **12% higher at 6 months** - Team review participation fell 30%

This is the empirical manifestation of Fowler's warning: "AI is an accelerator of whatever you already have. If best practices aren't in place, this velocity multiplier becomes a debt accelerator" [GitClear, 2025; Fowler, 2026].

5.4 The Gonzalez Experiment

Gabriel Gonzalez asked Claude Code to implement Symphony (OpenAI's autonomous coding framework) in Haskell from its specification document. The results: - Multiple bugs requiring manual intervention - The agent "spun silently without making progress" on tasks with no error messages - The specification was detailed but insufficient for reliable generation

His conclusion: "If you try to make a specification document precise enough to reliably generate a working implementation, you must necessarily contort the document into code or something strongly resembling code." This directly demonstrates that the spec-code convergence is not merely theoretical -- it manifests in practice whenever precision is demanded [Gonzalez, 2026].

5.5 Expert Positions on the Current Paradigm

Expert	Position	Key Quote/Insight
Kent Beck	AI augments, doesn't replace	TDD is "the strongest form of prompt engineering." AI works best when developers understand what they're building.
Martin Fowler	AI amplifies existing quality	"AI is an accelerator of whatever you already have." Vibe coding is not suitable for long-term applications.
Yann LeCun	LLMs fundamentally limited	LLMs are System 1 (pattern matching), not System 2 (reasoning). Will be "useless within five years."
Francois Chollet	Context-dependent	NL->code works for known patterns but fails for novel problems. The gap is small for templates, unbridgeable for novelty.
Chris Lattner	Better languages, not NL	"The most powerful languages will be ones that are expressive and readable." The future is better PLs, not NL replacement.
Rich Hickey	Simplicity first	The problem isn't NL->code translation but the complexity of what we're building. Simpler systems are easier for everyone.

Part VI: The Third Way -- Intermediate Representations and the Future of Harness Design

6.1 The Spectrum of Abstraction

Between natural language prose and executable code lies a rich landscape of intermediate representations:

```
Natural Language (English)
|
v [Low formalism, high ambiguity]
Structured Natural Language (Gherkin/BDD: Given-When-Then)
|
v [Moderate formalism, executable against tests]
Design by Contract (Preconditions/Postconditions)
|
v [Co-located with code, runtime-checkable]
Property-Based Specifications (QuickCheck properties)
|
v [Universal quantification, random testing]
Lightweight Model Checking (Alloy, Quint)
|
v [Automated bounded verification]
Formal Specification (TLA+, Z)
|
v [Full mathematical specification]
Rich Type Systems (Refinement types, Session types)
|
v [Compiler-verified properties]
Dependent Types (Agda, Idris, Lean)
|
v [Types = Propositions, Programs = Proofs]
Executable Code (Python, Rust, Haskell)
```

Each level trades accessibility for precision. The key insight from the lightweight formal methods movement: **partial application of formal methods at critical points yields most of the benefit at a fraction of the cost** [AWS/TLA+, Newcombe et al., 2015].

6.2 The Vericoding Pattern

The most promising emerging pattern -- "vericoding" (POPL 2026) -- combines all three elements:

1. **Human writes formal specification** (preconditions, postconditions, invariants)
2. **AI generates implementation code**
3. **Formal verifier checks** code against specification

4. **AI iterates** if verification fails

Current success rates: Dafny (82%), Verus/Rust (44%), Lean (27%). The 82% on Dafny demonstrates this is practically achievable today. The architecture neutralizes hallucination risk: "it doesn't matter if [AI] hallucinates nonsense" in proof attempts -- the proof checker catches invalid proofs, while valid proofs provide genuine guarantees [VeriCoding, 2025; Kleppmann, 2025].

6.3 AI as the Formalism Translator

Martin Kleppmann's December 2025 prediction that AI will make formal verification mainstream rests on three convergences:

1. **Cost reduction.** The seL4 microkernel required 20 person-years for 8,700 lines of C. AI could collapse this by orders of magnitude.
2. **New necessity.** AI-generated code *needs* formal verification because human review cannot scale to the volume.
3. **Inherent compatibility.** Proof checkers are small, verified codebases that provide an ideal oracle for LLM outputs.

The vision: developers specify properties in a high-level, declarative way; AI handles both implementation and proof. This mirrors how "we don't bother looking at machine code generated by a compiler" -- the formal guarantee replaces inspection [Kleppmann, 2025; Congdon, 2025].

6.4 What Lean 4 and AlphaProof Demonstrate

Lean 4 has become the focal point of AI-assisted formal verification: - **AlphaProof (DeepMind, 2024):** IMO silver-medal level proofs in Lean 4 - **Multiple systems (2025):** Gold-medal equivalent with formally verified Lean proofs - **DeepSeek-Prover-V2:** 88.9% on MiniF2F benchmark - **Harmonic AI:** \$100M raised for "hallucination-free" AI using Lean 4

This demonstrates that the boundary between specification and proof is dissolving. AI can traverse it in both directions -- generating proofs from specs and specs from informal descriptions [Lean 4, mathlib].

Part VII: Synthesis -- Implications for Harness and System Design

7.1 The Seven Answers to Gonzalez's Question

Each tradition answers "Is a sufficiently detailed spec code?" differently:

Thinker	Answer	Implication
Martin-Lof	Yes, by theorem	Types = specs = propositions. Programs = proofs.
Dijkstra	Yes, obviously	That is why formal notation is essential.
Brooks	Yes, and that's why specification is the hard part	The "hard part" cannot be automated away.
Parnas	Yes, but that's a failure of the spec	Information hiding requires the gap.
Kay	The question is wrong	Spec and code co-evolve through exploration.
Kolmogorov	Yes, and the spec is as long as the code	For incompressible programs, convergence is inevitable.
Rice	Yes, but you can't verify it	Verification of conformance is undecidable in general.

7.2 The Emerging Architecture for Future Harnesses

The research converges on a multi-layer architecture:

```

Layer 1: HUMAN INTENT (English + domain knowledge)
  |
  | [AI Formalization Layer – translates NL to formal spec]
  v
Layer 2: FORMAL SPECIFICATION (TLA+ / Dafny / Lean types / Contracts)
  |
  | [Human review of specification – the critical checkpoint]
  v
Layer 3: AI IMPLEMENTATION (code generation + proof generation)
  |
  | [Formal Verifier – deterministic correctness check]
  v
Layer 4: VERIFIED IMPLEMENTATION
  |
  | [Continuous verification – property-based tests, contracts]
  v
Layer 5: DEPLOYED SYSTEM

```

This architecture preserves human authority at the specification level (Layer 2) while delegating implementation and verification to AI. The formal specification is the key artifact -- the contract between human and machine, and the oracle for verification.

7.3 Six Design Principles for Harnesses

Drawing from all four research traditions, we derive six principles:

Principle 1: Operate at the Specification Level, Not the Code Level. Lamport, Jackson, Meyer, and the vericoding movement agree: the human's primary artifact should be a specification. The harness should encourage and support specification authoring, not just code editing. As Lamport argues: "the hardest part of writing a specification is choosing the proper abstraction."

Principle 2: Support Multiple Levels of Formalism. Not every part of a system needs the same rigor. A harness should support: - Gherkin/BDD for acceptance criteria (stakeholder-facing) - Contracts (preconditions/postconditions) for module interfaces - TLA+ or Quint for distributed protocol logic - Dependent types or refinement types for critical data structures - Property-based specifications for behavioral invariants

AWS's experience validates this: TLA+ for distributed protocols (subtle, catastrophic bugs), informal specs for UI code (visible, cheap bugs).

Principle 3: Use AI as a Formalism Translator. The historical barrier to formal methods was cost. AI can serve as the translator -- accepting informal intent and producing formal specifications for human review. Reviewing a spec is "vastly easier and quicker than writing" one from scratch (Kleppmann). This shifts the human role from "specification author" to "specification reviewer."

Principle 4: Employ Automatic, Continuous Verification. Proof checkers, model checkers, and property-based test generators should run automatically on every change. AI-generated proofs validated by deterministic checkers neutralize hallucination risk. The verification step is what makes the vericoding pattern trustworthy.

Principle 5: Preserve the Theory. Naur's insight that programs embody theories suggests harnesses should capture reasoning, not just decisions. Conversation logs with AI, design rationale, specification evolution histories, and commit messages are all partial externalizations of the theory. A good harness curates and makes searchable this theory-level knowledge across sessions and contributors.

Principle 6: Support Visual and Textual Formalisms. Harel's statecharts demonstrate that formal specification need not be textual. Harnesses should support visual specifications (state machines, sequence diagrams, data flow graphs) as first-class artifacts alongside textual ones. XState's adoption in the JavaScript ecosystem proves this can work in practice.

7.4 The Role Shift: From Programmer to Specification Reviewer

The consensus across nearly all positions -- formalist, pragmatist, and contrarian -- is that the human role is shifting. The question is *what it shifts to*:

- **Karpathy/Altman:** Humans become "intent specifiers" in natural language
- **Lamport/Dijkstra:** Humans become "specification authors" in formal language
- **Kleppmann/vericoding:** Humans become "specification reviewers" of AI-generated formal specs
- **Naur/Kay:** Humans remain "theory holders" -- the irreducible carriers of tacit knowledge about what the system should be

The most likely near-term trajectory combines all four: humans specify intent in NL, AI drafts formal specifications, humans review and refine the specs, AI generates implementations and proofs, formal tools verify, and humans hold the overarching theory of what the system is for and why.

Part VIII: Open Questions

1. **The education bottleneck.** Congdon identifies that formal verification experts could "fit in a large schoolbus." If the harness operates at the specification level, who has the skills to review specifications? This is the critical adoption barrier.
2. **The Naur challenge.** If programs embody tacit theories that cannot be fully externalized, then no harness architecture -- however sophisticated -- can fully bridge the abstraction gap. The theory will always partially reside in human minds.
3. **LeCun's architectural critique.** If LLMs lack genuine reasoning (System 2), then the NL->code gap is not a solvable engineering problem but a fundamental architectural limitation. Next-generation models (world models, neurosymbolic systems) may be required.
4. **The democratization tension.** Lowering the barrier to programming via NL risks creating a class of "vibe coders" who produce software they cannot maintain, debug, or understand. Nardi's spreadsheet precedent suggests this is manageable but not trivial -- spreadsheet errors cause significant real-world damage.
5. **Correlated errors.** When AI generates both code and tests, the same blind spots appear in both. The "homogenisation trap" (2025) means self-verified AI code has correlated errors that human review with *different* failure patterns must catch.

6. **The convergence timeline.** Vericoding achieves 82% on Dafny today. When it reaches 99%+, the specification-vs-code debate becomes moot for practical purposes. The question is whether this happens in 2 years or 20.

Source Index

Primary Texts (CS Giants)

1. Dijkstra, E.W. "On the foolishness of 'natural language programming'" (EWD 667), 1978. <https://www.cs.utexas.edu/~EWD/transcriptions/EWD06xx/EWD667.html>
2. Dijkstra, E.W. "On the cruelty of really teaching computing science" (EWD 1036), 1988
3. Dijkstra, E.W. "Go To Statement Considered Harmful." CACM 11(3), 1968
4. Dijkstra, E.W. "The Humble Programmer." CACM 15(10), 1972 (Turing Award)
5. Hoare, C.A.R. "An Axiomatic Basis for Computer Programming." CACM 12(10), 1969
6. Hoare, C.A.R. "The Emperor's Old Clothes." CACM 24(2), 1981 (Turing Award)
7. Knuth, D.E. "Literate Programming." The Computer Journal 27(2), 1984
8. Brooks, F.P. "No Silver Bullet." IFIP, 1986
9. Parnas, D.L. "On the Criteria To Be Used in Decomposing Systems into Modules." CACM 15(12), 1972
10. Parnas, D.L. and Clements, P.C. "A Rational Design Process." IEEE TSE SE-12(2), 1986
11. Turing, A.M. "On Computable Numbers." Proc. London Mathematical Society, 1936
12. Turing, A.M. "Computing Machinery and Intelligence." Mind 59(236), 1950
13. Church, A. "An Unsolvable Problem of Elementary Number Theory." AJM 58(2), 1936
14. Von Neumann, J. "First Draft of a Report on the EDVAC." 1945
15. Wirth, N. Algorithms + Data Structures = Programs. Prentice-Hall, 1975
16. Liskov, B. "The Power of Abstraction." Turing Award Lecture, 2009
17. Milner, R. "A Theory of Type Polymorphism in Programming." JCSS 17(3), 1978
18. Lamport, L., Merz, S., Newcombe, C. "The Future of TLA+." July 2024
19. Jackson, D. The Essence of Software. Princeton, 2021
20. Martin-Lof, P. "Constructive Mathematics and Computer Programming." 1979

Cognitive Science & HCI

1. Engelbart, D.C. "Augmenting Human Intellect." SRI Report, 1962

2. Licklider, J.C.R. "Man-Computer Symbiosis." IRE Trans., 1960
3. Kay, A.C. "A Personal Computer for Children of All Ages." ACM, 1972
4. Papert, S. Mindstorms. Basic Books, 1980
5. Shneiderman, B. Human-Centered AI. Oxford, 2022
6. Norman, D.A. The Design of Everyday Things. Basic Books, 1988
7. Suchman, L. Plans and Situated Actions. Cambridge, 1987
8. Nardi, B. A Small Matter of Programming. MIT Press, 1993
9. Chomsky, N. Syntactic Structures. Mouton, 1957
10. Norvig, P. "On Chomsky and the Two Cultures of Statistical Learning." 2011
11. Pinker, S. The Language Instinct. William Morrow, 1994
12. Miller, G.A. "The Magical Number Seven." Psychological Review, 1956
13. Kahneman, D. Thinking, Fast and Slow. FSG, 2011
14. Whorf, B.L. Language, Thought, and Reality. MIT Press, 1956

Information Theory & Formal Foundations

1. Kolmogorov, A.N. "Three Approaches to the Quantitative Definition of Information." 1965
2. Chaitin, G.J. "On the Length of Programs for Computing Finite Binary Sequences." JACM, 1966
3. Rice, H.G. "Classes of Recursively Enumerable Sets." Trans. AMS, 1953
4. Howard, W.A. "The Formulae-as-Types Notion of Construction." 1969
5. Wadler, P. "Propositions as Types." CACM 58(12), 2015
6. Scott, D.S. "Outline of a Mathematical Theory of Computation." Oxford, 1970
7. Naur, P. "Programming as Theory Building." 1985
8. Iverson, K.E. "Notation as a Tool of Thought." CACM, 1980 (Turing Award)

Current AI Tools & Empirical Evidence

1. Gonzalez, G. "A Sufficiently Detailed Spec is Code." Haskell for All, March 2026
2. METR. "Many SWE-bench-Passing PRs Would Not Be Merged." March 2026
3. GitClear. "AI Copilot Code Quality 2025." 2025
4. Karpathy, A. "Software 2.0." Medium, 2017; "Vibe Coding." X, February 2025
5. Fowler, M. "Fragments." February 2026; AI Coding Tags
6. Chollet, F. ARC Prize analysis; LLM reasoning limitations (LessWrong)
7. Lattner, C. Pragmatic Engineer interview; Latent Space interview, 2025
8. Beck, K. TDD, AI Agents and Coding. Pragmatic Engineer

Contrarian & Philosophical Sources

1. Altman, S. "The Intelligence Age." September 2024
2. Amodei, D. "Machines of Loving Grace." October 2024
3. Sutton, R. "The Bitter Lesson." 2019
4. Wittgenstein, L. Philosophical Investigations. 1953
5. Austin, J.L. How to Do Things with Words. 1962
6. Searle, J. "Minds, Brains, and Programs." 1980
7. Clark, A. Natural-Born Cyborgs. 2003
8. Hutchins, E. Cognition in the Wild. MIT Press, 1995
9. Vygotsky, L. Zone of Proximal Development framework
10. Lakoff, G. Philosophy in the Flesh. 1999
11. Brooker, M. "On the success of 'natural language programming'." December 2025
12. Syme, D. "On Natural Language Programming." August 2025
13. Litt, G. "Malleable Software in the Age of LLMs." March 2023

Formal Methods & Alternative Paradigms

1. Newcombe, C. et al. "How Amazon Web Services Uses Formal Methods." CACM, 2015
2. Kleppmann, M. "AI will make formal verification go mainstream." December 2025
3. Congdon, B. "The Coming Need for Formal Specification." December 2025
4. "A Benchmark for Vericoding." POPL 2026 (arXiv 2509.22908)
5. Brady, E. Type-Driven Development with Idris. Manning, 2017
6. Harper, R. Practical Foundations for Programming Languages. Cambridge, 2016
7. Harel, D. "Statecharts: A Visual Formalism." Science of Computer Programming, 1987
8. Meyer, B. "Design by Contract." ETH Zurich
9. Hoare, C.A.R. and He, J. Unifying Theories of Programming. 1998